

Experiment ‘p_13_7_recursive_index_of’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_7_recursive_index_of

Natural language statement: null

Method signature: p_13_7_recursive_index_of(text: string, str: string) returns (index: int)

Ensures

- `index == find_index(text, str, 0)`

Requires

- `1 <= |str| <= |text|`

Functional Code Given

```
function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_7_recursive_index_of(text: string, str: string) returns (index: int)`

The method should respect the following contract:

`requires 1 <= |str| <= |text|, ensures index == find_index(text, str, 0)`

The contract uses the following dafny code:

```
function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnOzFuBfli8S4bCEZMo6Y3QoEiK9Q

Model: gpt-5.5-2026-04-23

Created at: 1780666493

Finish reason: stop

Usage: 694 tokens in, and 9054 tokens out

```

{
  var pos: nat := 0;
  assert text[0..] == text;

  while pos + |str| <= |text| && text[pos..pos + |str|] != str
    invariant pos <= |text|
    invariant 1 <= |str| <= |text|
    invariant find_index(text, str, 0) == find_index(text[pos..], str, pos)
    decreases |text| - pos
  {
    ghost var s := text[pos..];
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
    assert s[..|str|] != str;

    if starts_with(s, str) {
      assert |str| <= |s| && forall k :: 0 <= k < |str| ==> s[k] == str[k];
      forall k | 0 <= k < |str|
        ensures s[..|str|][k] == str[k]
      {
        assert s[..|str|][k] == s[k];
      }
      assert |s[..|str|]| == |str|;
      assert s[..|str|] == str;
      assert false;
    }
    assert !starts_with(s, str);
    assert find_index(s, str, pos) == find_index(s[1..], str, pos + 1);
    assert s[1..] == text[pos + 1..];
    assert find_index(s, str, pos) == find_index(text[pos + 1..], str, pos + 1);
    assert find_index(text, str, 0) == find_index(text[pos + 1..], str, pos + 1);
    assert pos < |text|;
    pos := pos + 1;
  }

  assert !(pos + |str| <= |text| && text[pos..pos + |str|] != str);
  ghost var s := text[pos..];

  if pos + |str| <= |text| {
    assert text[pos..pos + |str|] == str;
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
    assert s[..|str|] == str;
    forall k | 0 <= k < |str|
      ensures s[k] == str[k]
    {
      assert s[..|str|][k] == str[k];
      assert s[..|str|][k] == s[k];
    }
    assert forall k :: 0 <= k < |str| ==> s[k] == str[k];
    assert starts_with(s, str);
  }
}

```

```

    assert find_index(s, str, pos) == pos;
    index := pos;
  } else {
    assert pos + |str| > |text|;
    assert |s| == |text| - pos;
    assert |s| < |str|;
    assert find_index(s, str, pos) == -1;
    index := -1;
  }

  assert index == find_index(s, str, pos);
  assert find_index(text, str, 0) == find_index(s, str, pos);
  assert index == find_index(text, str, 0);
}

```

Final Program

```

function starts_with(text: string, str: string) : (b: bool)
  ensures b == (|str| <= |text| && forall i :: 0 <= i < |str| ==> text[i] == str[i])
{
  if |str| == 0 then true
  else if |text| == 0 then false
  else if text[0] == str[0] then starts_with(text[1..], str[1..])
  else false
}

function find_index(text: string, str: string, index: nat) : (ret : int)
  ensures starts_with(text, str) ==> ret == index
{
  if |text| < |str| then -1
  else if starts_with(text, str) then index
  else find_index(text[1..], str, index + 1)
}

method p_13_7_recursive_index_of(text: string, str: string) returns (index: int)
  requires 1 <= |str| <= |text|
  ensures index == find_index(text, str, 0)
{
  var pos: nat := 0;
  assert text[0..] == text;

  while pos + |str| <= |text| && text[pos..pos + |str|] != str
    invariant pos <= |text|
    invariant 1 <= |str| <= |text|
    invariant find_index(text, str, 0) == find_index(text[pos..], str, pos)
    decreases |text| - pos
  {
    ghost var s := text[pos..];
    assert |str| <= |s|;
    assert s[..|str|] == text[pos..pos + |str|];
  }
}

```

```

assert s[..|str|] != str;

if starts_with(s, str) {
  assert |str| <= |s| && forall k :: 0 <= k < |str| ==> s[k] == str[k];
  forall k | 0 <= k < |str|
    ensures s[..|str|][k] == str[k]
  {
    assert s[..|str|][k] == s[k];
  }
  assert |s[..|str|]| == |str|;
  assert s[..|str|] == str;
  assert false;
}
assert !starts_with(s, str);
assert find_index(s, str, pos) == find_index(s[1..], str, pos + 1);
assert s[1..] == text[pos + 1..];
assert find_index(s, str, pos) == find_index(text[pos + 1..], str, pos + 1);
assert find_index(text, str, 0) == find_index(text[pos + 1..], str, pos + 1);
assert pos < |text|;
pos := pos + 1;
}

assert !(pos + |str| <= |text| && text[pos..pos + |str|] != str);
ghost var s := text[pos..];

if pos + |str| <= |text| {
  assert text[pos..pos + |str|] == str;
  assert |str| <= |s|;
  assert s[..|str|] == text[pos..pos + |str|];
  assert s[..|str|] == str;
  forall k | 0 <= k < |str|
    ensures s[k] == str[k]
  {
    assert s[..|str|][k] == str[k];
    assert s[..|str|][k] == s[k];
  }
  assert forall k :: 0 <= k < |str| ==> s[k] == str[k];
  assert starts_with(s, str);
  assert find_index(s, str, pos) == pos;
  index := pos;
} else {
  assert pos + |str| > |text|;
  assert |s| == |text| - pos;
  assert |s| < |str|;
  assert find_index(s, str, pos) == -1;
  index := -1;
}

assert index == find_index(s, str, pos);
assert find_index(text, str, 0) == find_index(s, str, pos);
assert index == find_index(text, str, 0);

```

}

Total Token Usage

Input tokens: 694

Output tokens: 9054

Reasoning tokens: 8188

Sum of ‘total tokens’: 9748

Experiment Timings

Overall Experiment started at 1780666492215, ended at 1780666712744, lasting 220529ms (220.53 seconds)

Iteration #1 started at 1780666492249, ended at 1780666712744, lasting 220495ms (220.50 seconds)